

Target System

- Target System
 - Introduction
 - Learning Objectives
 - Procedure and Terminology
 - Program Structure
 - CT-Board Wiki
 - **Task 1:** Setup work your environment
 - **Task 2:** Write your own C program
 - Functions to read from and write to the memory
 - Reading the DIP switches and writing to the LEDs
 - **Task 3:** Extend the C program
 - Additional DIP switches
 - Control a seven segment display
 - Grading

Introduction

In this laboratory you will familiarise yourself with the CT Board and its input and output capabilities.

Learning Objectives

- You can compile a C program, link it and load the executable onto the target system
- You can utilise the input and output capabilities of the CT board in a program.
- You are able to write simple C programs for reading switches, controlling LEDs and a 7-segment display

Procedure and Terminology

For all our labs, we use a target system (CT-Board) based on a chip from the **STM32F4** microcontroller family. The PC (host) serves as a development system (editing, compiling and assembling etc.) to build an executable of the program. Next the generated executable is loaded through USB onto the CT Board.

Program Structure

The programs which you are going to write usually run in an infinite loop. E.g. changes on input devices (switch position, etc.) can only be detected through repetitive queries. This type of query is called polling. Therefore, the main program has to contain an infinite loop:

```
1 int main(void)
2 {
3     /* initialization */
4     while ( 1 )
5     {
6         /* application */
7     }
8 }
```

Of course, in real systems this does not always make sense, the processor is permanently active and consumes energy. Therefore, most processors have mechanisms with which they can go to sleep until they are woken up by an external event. This is a topic for CT2 though.

CT-Board Wiki

Most information about the CT-Board can be found on the [CT-Board Wiki](#). Here is an non-exhaustive list of information to be found on ennis:

- setting up a project for assembly or C
- GPIO assignment
- addresses and explanations (with examples) for all peripherals

Task 1: Setup work your environment

Visit [Ennis \(CT wiki\)](#) for instructions on how to install Keil.

Alternatively you can follow the guide in `../install-gnu-toolchain.md` to set up your system for a workflow without Keil.

Task 2: Write your own C program

The next step will be to read the switch states from the associated memory address and write the values to the associated LED address.

Functions to read from and write to the memory

The module `utils_ctboard` provides functions to access the memory. It contains interface declarations in the headerfile `./src/utils_ctboard.h` and the implementation in the `./src/utils_ctboard.c`.

Study the header file:

- Which functions are provided by the module?
- What parameters do the functions take?
- What types are returned by the functions?

Add the module `utils_ctboard` to your C-project.

Makefile (VSCode uses the Makefile)

- Add `utils_ctboard.c` to the variable `C_SRC` (list of file to compile, the path can be omitted, because of the variable `VPATH` in `../common/root.mk`). Expand the variable using `C_SRC += <source_file.c>`.
- Import the module into your program with an `#include` statement.

Keil

- Add the file `utils_ctboard.c` to your project by using the menu entry “Add Existing Files to ...”.
- `utils_ctboard.h` does not have to be added. Keil uVision only needs to know which `*.c` files belong to the project. The `*.h` files will be included in the `*.c` files through the `#include` statement.

Reading the DIP switches and writing to the LEDs

Write a program which repeatedly reads a single byte from the dip switches `SW[7..0]` (at the address `0x6000_0200`) and writes those values to `LED[7..0]` (at the address `0x6000_0100`).

- Include the functions of `utils_ctboard` in your `main.c` by using `#include "utils_ctboard.h"`.
- Which function is used to read a single byte and to write a single byte respectively?
- Define appropriate macros (`#define`) for the memory addresses, which you are using in the function call.
- Define an appropriate variable for the read value. Implement the output of this variable on an individual line. This simplifies debugging, as it allows you to step through the program line by line.

Create an executable of your program, run the debugger and start the program. Check the correct function of your program with different positions of the dip switches.

- Read the `../readme.md` for instructions on these steps.
- Familiarise yourself with the features of the debugger like *Single Stepping* and *Breakpoints*.
- Check out the Register View. It displays the contents of the processors registers at every step.

Task 3: Extend the C program

Additional DIP switches

Change your program to read the values of DIP switches `SW[31..0]` and display the values on the LEDs `LED[31..0]`.

- This can be achieved by reading and writing a word (32 bits) at once. The addresses stay the same.
- This task is an extension of the first program. No need to show the first task separately.

Check the correct function of your program with different positions of the DIP switches.

Control a seven segment display

Expand your program further. Read the value of rotary switch P11 from address `0x6000_0211` and output the read value on the seven-segment display DS0 at address `0x6000_0110`. The conversion to seven segment code shall be done in software and not using the provided hardware module.

- After reading the byte the value of the rotary switch will be located in the least significant nibble (bits [3..0]). Extract the value of the rotary switch through an AND-operation with a mask of `0x0F`. This removes the unused upper 4 bits.
- To control the seven-segment display, an array holding 16 elements of the type `const uint8_t` can be defined. The element with index 0 contains the bit pattern for the number 0 the element with index 1 contains the bit pattern for the number 1, etc. Determine the bit patterns accordingly. Keep in mind that the individual segments of the display are active-low. The value read from the rotary switch is used as an index to access the array.
- On the 7-segment display the uppercase letter B cannot be distinguished from the number 8. Likewise, the uppercase letter D cannot be distinguished from the number 0. Therefore, display the hex numbers as lower case b and d to allow a distinction.
- For debugging purposes, you can output the value read from address `0x6000_0211` on the LEDs at address `0x6000_0100`. In parallel, you can output the value of the seven-segment display (i.e. the value of the array access) on the LEDs at address `0x6000_0101`.

Verify the correct function with all possible positions of the rotary switch.

Grading

The working program has to be presented to the lecturer. The **individual** student has to understand the solution/source code and has to be able to explain it to the lecturer.

Criteria	Weight
Task 2	2 / 4
Task 3	2 / 4